



Local LLM Hackathon

Leveraging Local LLMs for Mobile Development

Technical Deep Dive

June 18, 2026

68 commits • 4 features • 100% local inference

Hackathon Overview

68

Commits Since 62fc4f19

4

Major Features

100%

Local Inference

Features Implemented

-  **Speech-to-Code:** WhatsApp-style voice coding with 650-1300ms latency
-  **Inline Suggestions:** Copilot-style ghost text completions in Sora editor
-  **Vector Search:** Semantic code search using embedding-based similarity
-  **ADB Broadcast Receiver:** Remote ILM control via ADB for

Three Pillars of Local LLM Integration



Speech-to-Code

Problem: Typing on mobile is slow

Solution: Voice commands → STT → LLM → code insertion

Key Tech: Moonshine STT, Android SpeechRecognizer, local LLM pipeline



Inline Suggestions

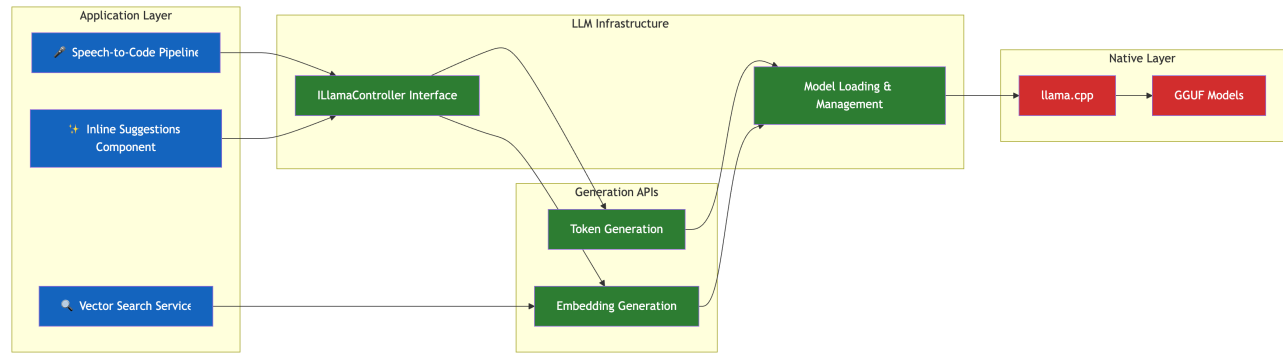
Problem: No autocomplete for complex patterns

Solution: Ghost text multi-line completions

Key Tech: Sora editor components, custom rendering, LLM caching



Unified Architecture: Shared LLM Core



All features leverage the same `ILlamaController` for inference

Benefit: Shared model loading, unified memory management

Speech-to-Code: WhatsApp-Style Voice Coding

Design Goals

- **Viral Appeal:** Familiar long-press pattern from WhatsApp
- **Trust Building:** Preview-before-insert prevents errors
- **Fast Execution:** 650-1300ms total latency (STT + LLM)
- **Safety:** Preview panel before code insertion

User Flow

1. Long-press mic button → recording overlay with waveform
2. Speak command: *"create a RecyclerView adapter for user list"*
3. Release button → processing (STT + Intent + LLM)
4. Preview panel shows transcription + generated code
5. Tap "Insert" → code appears with typing animation

Files: `SpeechToCodePipeline.kt` , `SpeechToCodeViewModel.kt` ,
`VoiceCodeAction.kt`

Speech-to-Code: Architecture



Key Components

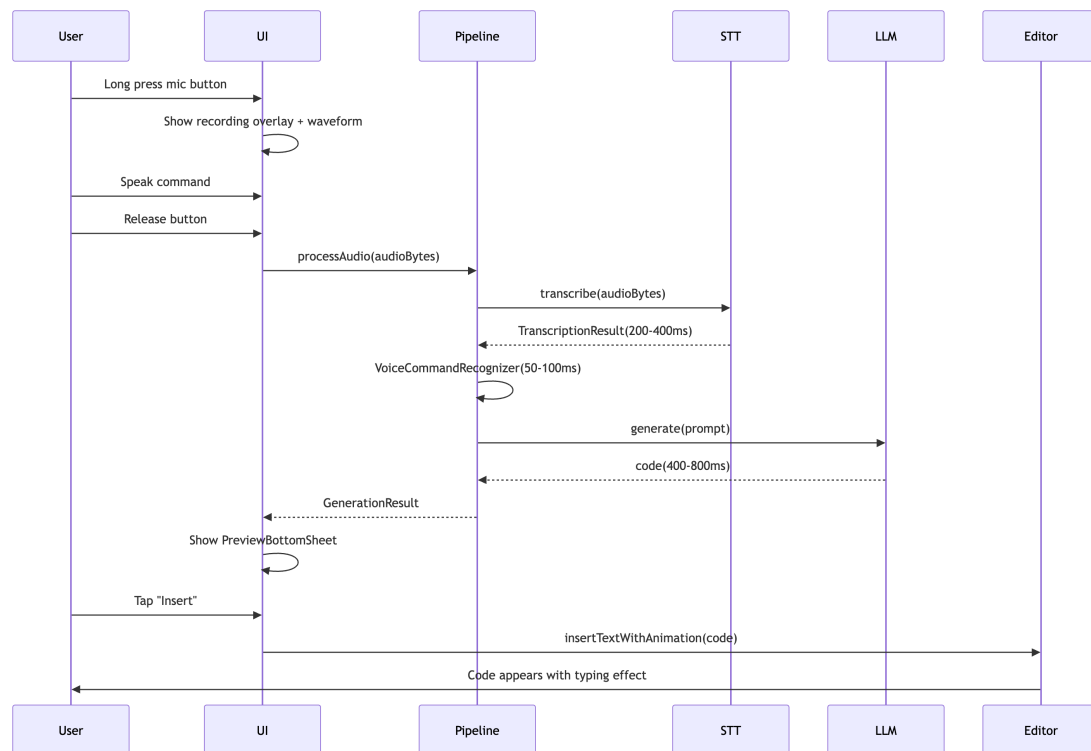
SpeechToCodePipeline:

- Coordinates STT → Intent → LLM
- Handles timeouts and errors
- Returns GenerationResult with latency

Dual STT Support:

- Cloud: Android SpeechRecognizer
- Offline: Moonshine (200-400ms)
- User preference in AI Settings

Speech-to-Code: Pipeline Flow



Total Latency: 650-1300ms (90th percentile)

Bottleneck: LLM generation (400-800ms)

Speech-to-Code: Performance & UX

Latency Breakdown (from SpeechToCodePipeline.kt)

200-400ms

Moonshine STT

50-100ms

Intent Recognition

400-800ms

LLM Code Generation

State Machine

```
graph LR
  IDLE --> RECORDING
  RECORDING --> PROCESSING
  PROCESSING --> PREVIEW
  PREVIEW --> INSERT_CANCEL["(INSERT | CANCEL)"]
  INSERT_CANCEL --> IDLE
  PREVIEW --> ERROR
  ERROR --> TOAST
  TOAST --> IDLE
```

Key Achievements

✨ Inline Suggestions: Copilot-Style Completions

Design Goals

- **Seamless UX:** Ghost text appears automatically after 3 characters + 300ms idle
- **Manual Control:** Ctrl+Space or toolbar button for on-demand suggestions
- **Multi-line Support:** 3-5 lines of contextual code completion
- **Local Performance:** Optimized for 4GB devices with caching

User Interaction

- **Trigger:** Auto (3 chars + 300ms) or Manual (Ctrl+Space)
- **Display:** Semi-transparent gray ghost text (#808080 @ 40% opacity)
- **Accept:** Tab key inserts entire suggestion
- **Dismiss:** Esc key or continue typing incompatible characters

Implementation: Extends Sora editor via

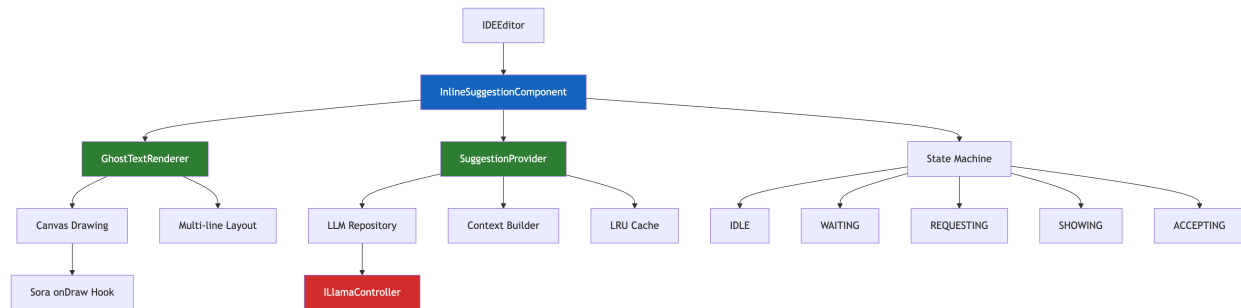
```
EditorBuiltinComponent
```

Inline Suggestions: Pipeline Flow



Performance: 300-600ms from trigger to display

Inline Suggestions: Sora Component Architecture



Key Insight: Follows Sora's component pattern
Renders ghost text via custom Canvas drawing

Inline Suggestions: Ghost Text Rendering

GhostTextRenderer Implementation

```
class GhostTextRenderer(private val editor: IDEEditor) {  
    private val ghostPaint: Paint = Paint().apply {  
        color = Color.argb(102, 128, 128, 128) // 40% opacity  
        textSize = editor.textSizePx  
        typeface = editor.typefaceText  
    }  
  
    fun drawSuggestion(canvas: Canvas, suggestion: SuggestionData) {  
        // 1. Calculate cursor position in screen coordinates  
        // 2. Split multi-line suggestions  
        // 3. Render each line with proper indentation  
        // 4. Clip to visible region  
    }  
}
```

Rendering Flow

1. Hook into Sora's `onDraw` lifecycle
2. Calculate ghost text position from cursor coordinates
3. Measure text using editor's font/size
4. Draw each line with vertical offset
5. Apply transparency and clipping

Inline Suggestions: LLM Integration & Caching

SuggestionProvider Strategy

Context Building:

```
data class SuggestionContext(  
    val fileContent: String,  
    val cursorPosition: Position,  
    val language: String,  
    val previousLines: List,  
    val currentLinePrefix: String  
)
```

Caching:

- LRU cache: 20 recent suggestions
- Key: Hash(file, cursor, context)
- Expiry: 30s or on file edit
- Prevents redundant LLM calls

Performance Optimizations

- **Debouncing:** 300ms idle before LLM request
- **Cancellation:** Cancel in-flight requests on new input

Vector Search: Semantic Code Search

The Problem with Keyword Search

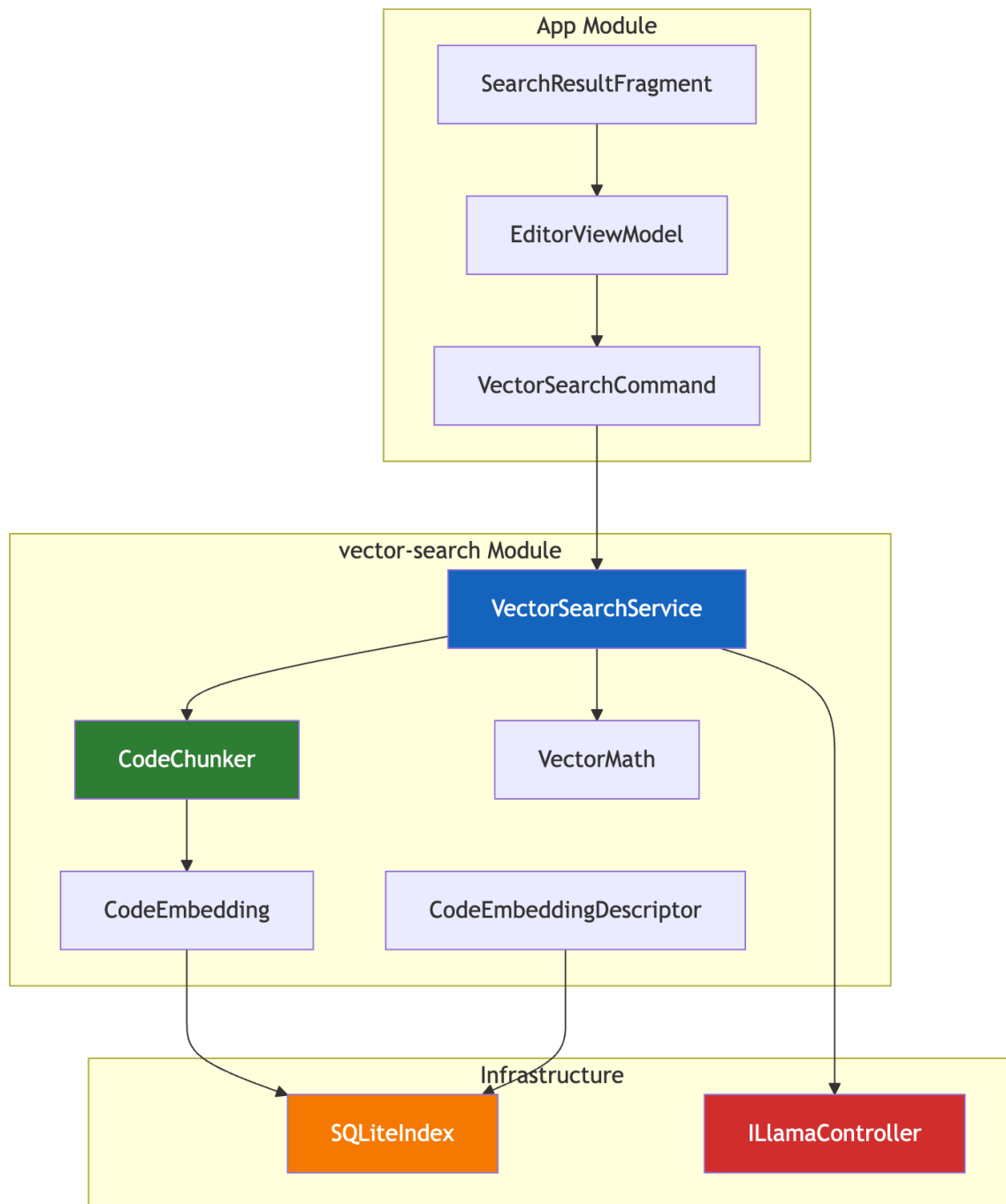
- Search "authentication" → misses "login", "signin", "verify user"
- Search "database" → misses "repository", "DAO", "data access"
- No understanding of semantic similarity

Embedding-Based Solution

- **Code Chunking:** Split files into 200-500 char semantic chunks
- **Embedding Generation:** Convert chunks to 384-dim vectors via local LLM
- **Similarity Search:** Cosine similarity between query embedding and code embeddings
- **On-Demand Indexing:** Index project on first search, cache in SQLite

Result: Search "auth" finds authentication logic even without exact keyword match

Vector Search: Module Architecture



Module Structure: Standalone `vector-search` library

Dependencies: `llama-api` , `lsp:indexing`

Vector Search: On-Demand Indexing



Vector Search: Search Pipeline

VectorSearchService.search() Implementation

```
suspend fun search(  
    query: String,  
    limit: Int = 20,  
    threshold: Float = 0.0f  
): List {  
    return withContext(Dispatchers.Default) {  
        // 1. Generate embedding for query  
        val queryEmbedding = llamaController.generateEmbedding(query)  
  
        // 2. Load all code embeddings from SQLite  
        val allEmbeddings = index.query(IndexQuery.ALL)  
  
        // 3. Calculate cosine similarity for each  
        allEmbeddings.map { codeEmbedding ->  
            val similarity = VectorMath.cosineSimilarity(  
                queryEmbedding,  
                codeEmbedding.embedding  
            )  
            Pair(codeEmbedding, similarity)  
        }  
        // 4. Filter by threshold, sort, take top K  
        .filter { (_, score) -> score >= threshold }  
        .sortedByDescending { (_, score) -> score }  
        .take(limit)  
        .map { (embedding, _) -> embedding }  
    }  
}
```

Performance: 100-500ms for 1000 embeddings (cosine similarity is



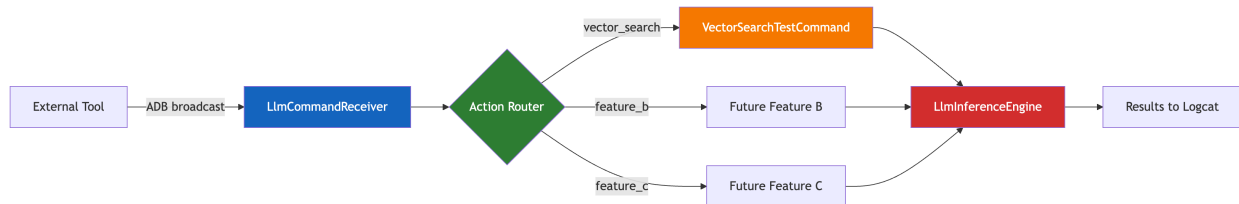
ADB Broadcast Receiver: Remote LLM Control

Problem & Solution

Problem: Need to test and automate LLM features externally

Solution: BroadcastReceiver that exposes LLM features via ADB commands

Architecture



Usage Example

```
adb shell am broadcast \
  -a com.itsaky.androidide.LLM_COMMAND \
  --es action "vector_search" \
  --es query "authentication"
```

Key Features

- **Extensible:** Easy to add new actions/features
- **Auto model loading:** Loads LLM model if needed
- **Detailed logging:** Results output to logcat
- **CRUD operations:** Enables Create, Read, Update via broadcast



Technical Achievements

68

Commits

4

Major Features

100%

Local Inference

Key Innovations



Speech-to-Code:

- Dual STT (Cloud + Moonshine)
- Preview-before-insert UX
- Spanish support
- Sub-second latency



Inline Suggestions:

- Custom Sora components
- Ghost text rendering
- LRU caching
- Multi-line completions



Vector Search:

- Standalone vector-search module

• On-demand indexing

Questions?

Key Repositories

CodeOnTheGo: Android IDE with local LLM integration

Commits: 62fc4f19..HEAD (68 commits)

Date Range: June 18, 2026

Thank you!

 Local LLMs on Mobile • 100% On-Device Inference • Zero Cloud Dependencies